

QCM Intelligence Artificielle

Voici les corrigés des deux examens que je t'ai proposés :

✓ CORRIGÉ — EXAMEN 1 : Recuit simulé

■ PARTIE I — QCM (10 points)

1. ✓ Une technologie permettant de simuler l'intelligence humaine
 2. ✓ Une technique pour que l'ordinateur apprenne à partir de données
 3. ✓ Problème du voyageur de commerce (TSP)
 4. ✓ Elle accepte parfois des solutions moins bonnes pour explorer davantage
 5. ✓ D'introduire de la diversité dans la population
 6. ✓ Une solution codée du problème
 7. ✓ L'usage de réseaux de neurones profonds
 8. ✓ Converge vers un minimum en suivant le gradient
✓ Est basé sur la dérivée de la fonction coût
 9. ✓ D'accepter ou non une solution selon une probabilité dépendante de la température
 10. ✓ Elle diminue progressivement pour stabiliser la solution
-

Total QCM : 10/10 si tout est juste

🔧 PARTIE II — Exercice (Recuit simulé sur TSP)

Résumé des réponses attendues :

1. Fonction de distance totale :

```
python
```

```
import numpy as np

def total_distance(x, y, chemin):
    dist = 0
    for i in range(len(chemin)):
        j = (i + 1) % len(chemin)
        dx = x[chemin[i]] - x[chemin[j]]
        dy = y[chemin[i]] - y[chemin[j]]
        dist += np.sqrt(dx**2 + dy**2)
    return dist
```

2. Génération de voisin (swap de deux villes) :

python

```
import random

def generate_neighbor(chemin):
    a, b = random.sample(range(len(chemin)), 2)
    voisin = chemin.copy()
    voisin[a], voisin[b] = voisin[b], voisin[a]
    return voisin
```

3. Recuit simulé (simplifié) :

python

```
def simulated_annealing(x, y, chemin_initial, T_init=100, alpha=0.95,
iterations=1000):
    import math
    chemin = chemin_initial.copy()
    best = chemin.copy()
    T = T_init
    for _ in range(iterations):
        voisin = generate_neighbor(chemin)
        dE = total_distance(x, y, voisin) - total_distance(x, y, chemin)
        if dE < 0 or random.random() < math.exp(-dE / T):
            chemin = voisin
            if total_distance(x, y, chemin) < total_distance(x, y, best):
                best = chemin
```

```
T *= alpha  
return best
```

✓ CORRIGÉ — EXAMEN 2 : Algorithmes génétiques

■ PARTIE I — QCM (10 points)

1. ✓ Traiter des images, sons ou vidéos
2. ✓ L'optimisation combinatoire travaille sur un ensemble fini et discret
3. ✓ Les problèmes NP-difficiles où les méthodes classiques échouent
4. ✓ Algorithmes de colonies de fourmis
5. ✓ Difficile à résoudre en temps raisonnable quand la taille du problème augmente
6. ✓ $\forall s \in S, f(s^*) \leq f(s)$
7. ✓ Sélection parmi un sous-ensemble d'individus les plus adaptés
✓ Est moins influencé par les valeurs extrêmes que la roulette
8. ✓ Elle augmente la diversité génétique
9. ✓ Une population de solutions qui évolue
10. ✓ Évaluer la qualité/adaptation d'un individu

Total QCM : 10/10 si tout est juste

✏ PARTIE II — Exercice (AG minimisation)

1. Fonction de fitness :

```
python  
  
def fitness(individu):  
    return sum(individu)  # car on veut minimiser le nombre de 1
```

2. Sélection par tournoi :

python

```
def tournoi_selection(population, k=2):
    from random import sample
    candidats = sample(population, k)
    return min(candidats, key=fitness)
```

3. Croisement à un point :

python

```
def croisement(p1, p2):
    point = random.randint(1, len(p1) - 1)
    enfant1 = p1[:point] + p2[point:]
    enfant2 = p2[:point] + p1[point:]
    return enfant1, enfant2
```

4. Mutation (0.2 par bit) :

python

```
def mutation(individu, prob=0.2):
    return [bit if random.random() > prob else 1 - bit for bit in individu]
```

5. Génération et affichage :

python

```
def generation_suivante(population):
    nouvelle_population = []
    while len(nouvelle_population) < len(population):
        p1 = tournoi_selection(population)
        p2 = tournoi_selection(population)
        e1, e2 = croisement(p1, p2)
        nouvelle_population.append(mutation(e1))
        if len(nouvelle_population) < len(population):
            nouvelle_population.append(mutation(e2))
    return nouvelle_population
```

Exemple d'utilisation :

```
population = [  
    [1, 0, 0, 1, 1, 1],  
    [0, 1, 0, 0, 1, 0],  
    [1, 1, 1, 0, 1, 1],  
    [1, 0, 1, 1, 0, 0],  
    [0, 0, 1, 0, 0, 0],  
    [0, 1, 0, 1, 1, 1]  
]  
  
for gen in range(10):  
    population = generation_suivante(population)  
    best = min(population, key=fitness)  
    print(f"Génération {gen+1} - Meilleur individu : {best}, Score :  
    {fitness(best)}")
```

Souhaitez-tu que je crée un **PDF complet d'examen avec corrigé** prêt à imprimer ?